

Patroni — Automated PostgreSQL HA

Table of Contents

1. [Learning Objectives](#)
 2. [Patroni Architecture](#)
 3. [DCS: etcd, Consul, ZooKeeper](#)
 4. [Installation](#)
 5. [Complete patroni.yml Configuration](#)
 6. [patronictl Commands](#)
 7. [REST API](#)
 8. [Bootstrap and Initialization](#)
 9. [Failover and Switchover](#)
 10. [Monitoring Patroni](#)
 11. [Patroni with PgBouncer and HAProxy](#)
 12. [Common Mistakes](#)
 13. [Best Practices](#)
 14. [Performance Considerations](#)
 15. [Interview Questions](#)
 16. [Exercises and Solutions](#)
-

Learning Objectives

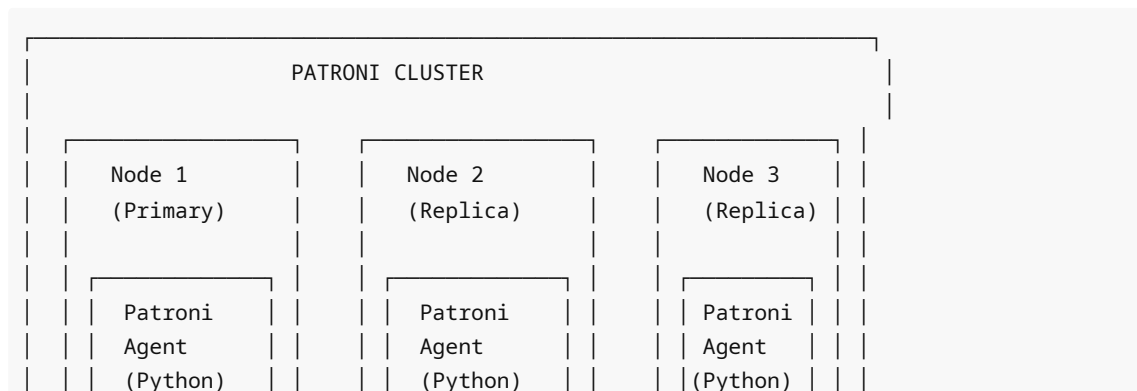
By the end of this module, you will be able to:

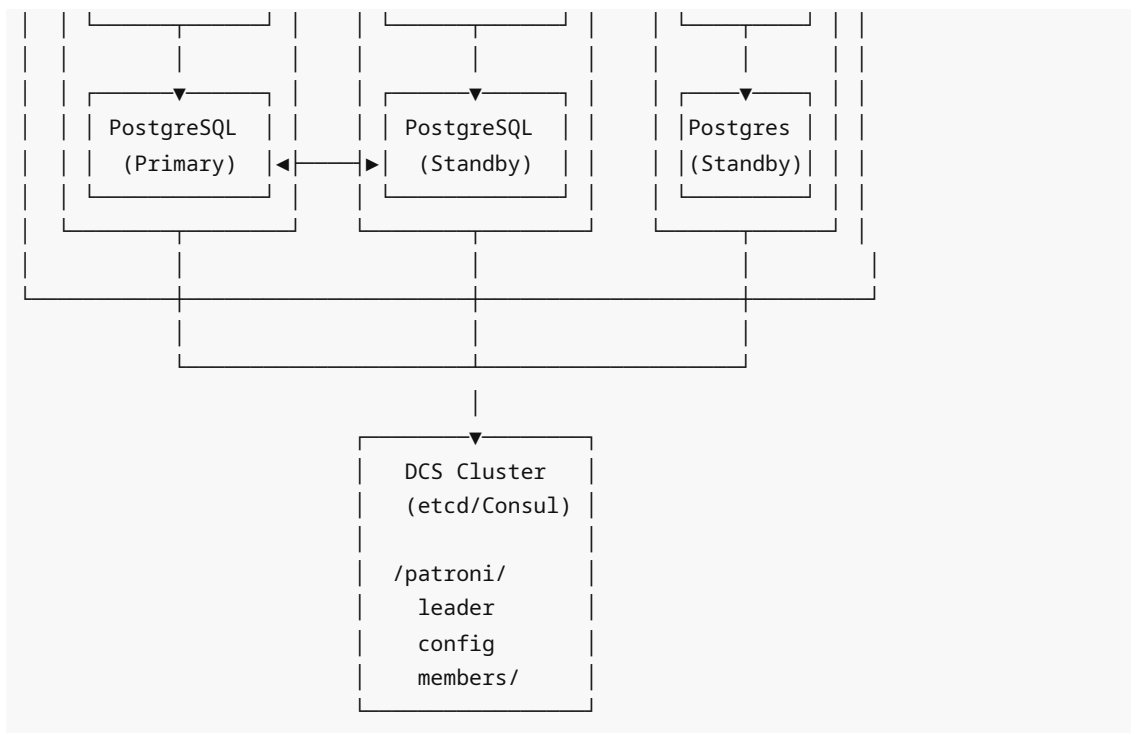
- Explain Patroni's architecture and how it achieves automatic HA
 - Set up a 3-node Patroni cluster with etcd
 - Write a production-grade patroni.yml configuration
 - Use patronictl for cluster management
 - Integrate Patroni with HAProxy and PgBouncer
 - Troubleshoot common Patroni issues
-

Patroni Architecture

Patroni is a Python-based HA solution for PostgreSQL that uses a **Distributed Configuration Store (DCS)** to manage leader election and cluster state.

Core Components





How Patroni Prevents Split-Brain

Leader Lock Mechanism:

1. Only the leader holds a "lock" (key) in DCS with a TTL (e.g., 30 seconds)
2. Leader MUST renew the lock every $\text{ttl}/2$ seconds
3. If leader can't reach DCS → cannot renew lock → SELF-DEMOTES (stops PostgreSQL writes)
4. A replica wins the next election (creates the lock key in DCS)
5. Only the DCS lock holder can be primary

No manual fencing needed – leader self-demotes on DCS loss!

DCS: etcd, Consul, ZooKeeper

DCS	Pros	Cons	Port
etcd	Native k8s, simple, fast	Must run separate cluster	2379
Consul	Service discovery + DCS, good UI	More complex setup	8500
ZooKeeper	Battle-tested	Java, complex, less common for PG	2181
etcd (kubernetes)	Built-in k8s	k8s dependency	—

Minimal etcd Cluster Setup

```
# 3-node etcd cluster (run on 3 separate nodes)
# Node 1: 192.168.1.10, Node 2: 192.168.1.11, Node 3: 192.168.1.12
```

```
# Install etcd
sudo apt-get install -y etcd

# /etc/etcd/etcd.conf on Node 1:
cat > /etc/etcd/etcd.conf << 'EOF'
ETCD_NAME=etcd1
ETCD_DATA_DIR=/var/lib/etcd
ETCD_LISTEN_PEER_URLS=http://192.168.1.10:2380
ETCD_LISTEN_CLIENT_URLS=http://192.168.1.10:2379,http://127.0.0.1:2379
ETCD_INITIAL_ADVERTISE_PEER_URLS=http://192.168.1.10:2380
ETCD_ADVERTISE_CLIENT_URLS=http://192.168.1.10:2379
ETCD_INITIAL_CLUSTER=etcd1=http://192.168.1.10:2380,etcd2=http://192.168.1.11:2380,etcd3=http://192.168.1.12:2380
ETCD_INITIAL_CLUSTER_STATE=new
ETCD_INITIAL_CLUSTER_TOKEN=patroni-etcd-cluster
EOF

sudo systemctl enable etcd
sudo systemctl start etcd

# Verify etcd cluster health
etcdctl endpoint health --
endpoints=http://192.168.1.10:2379,http://192.168.1.11:2379,http://192.168.1.12:2379
```

Installation

```
# Install Patroni and dependencies on each PostgreSQL node

# Method 1: pip
sudo apt-get install -y python3-pip python3-dev libpq-dev
pip3 install patroni[etcd] # for etcd DCS
# OR
pip3 install patroni[consul] # for Consul
# OR
pip3 install patroni[zookeeper] # for ZooKeeper

# Method 2: Package manager (Ubuntu)
sudo apt-get install -y patroni

# Verify
patroni --version
patronictl version
```

Complete patroni.yml Configuration

```
# /etc/patroni/patroni.yml
# Complete production configuration for Node 1
```

```

scope: my-postgres-cluster      # Cluster name (must be same on all nodes)
namespace: /patroni/            # DCS key prefix
name: pg-node1                  # Unique name for THIS node

restapi:
  listen: 0.0.0.0:8008           # Patroni REST API listens here
  connect_address: 192.168.1.100:8008 # How others reach this node's API
  # Optional: authentication for REST API
  # authentication:
  #   username: patroni
  #   password: patroni_api_password

etcd3:                          # Use etcd3 API (etcd3 preferred over etcd)
  hosts: 192.168.1.10:2379,192.168.1.11:2379,192.168.1.12:2379
  # If using single etcd node (dev only):
  # host: 127.0.0.1:2379

# For Consul DCS instead:
# consul:
#   host: 127.0.0.1:8500
#   url: http://127.0.0.1:8500

bootstrap:
  # Bootstrapping configuration (used only when initializing a NEW cluster)
  dcs:
    ttl: 30                      # Leader lock TTL in seconds
    loop_wait: 10                # How often Patroni checks DCS (seconds)
    retry_timeout: 10            # DCS operation timeout
    maximum_lag_on_failover: 1048576 # Max lag in bytes to be eligible for
    promotion (1MB)
    # Master can only be elected if its lag < maximum_lag_on_failover

  synchronous_mode: false       # Set true for synchronous replication enforcement
  # synchronous_mode_strict: false # If true, primary blocks if no sync standby

postgresql:
  use_pg_rewind: true           # Enable pg_rewind for fast re-sync
  use_slots: true               # Use replication slots
  parameters:
    # These parameters are applied cluster-wide via DCS
    max_connections: 200
    shared_buffers: 2GB
    effective_cache_size: 6GB
    work_mem: 16MB
    maintenance_work_mem: 512MB
    wal_level: replica
    max_wal_senders: 10
    max_replication_slots: 10
    wal_keep_size: 1GB
    wal_compression: 'on'
    wal_log_hints: 'on'         # Required for pg_rewind
    hot_standby: 'on'

```

```

hot_standby_feedback: 'on'
log_destination: stderr
logging_collector: 'on'
log_directory: /var/log/postgresql
log_filename: postgresql-%Y-%m-%d_%H%M%S.log
log_min_duration_statement: 1000 # Log queries > 1 second
shared_preload_libraries: pg_stat_statements,auto_explain
track_activity_query_size: 4096
track_commit_timestamp: 'on'
checkpoint_completion_target: 0.9
random_page_cost: 1.1
effective_io_concurrency: 200
autovacuum_max_workers: 5
autovacuum_vacuum_scale_factor: 0.01
autovacuum_analyze_scale_factor: 0.005

# pg_hba.conf entries written at bootstrap
pg_hba:
- host replication replicator 0.0.0.0/0 scram-sha-256
- host all all 0.0.0.0/0 scram-sha-256
- local all all peer
- host all all 127.0.0.1/32 scram-sha-256
- host all all ::1/128 scram-sha-256

# SQL run at bootstrap initialization (only once, on first node)
initdb:
- encoding: UTF8
- data-checksums # Highly recommended for data integrity

# Post-bootstrap SQL scripts
post_bootstrap: /etc/patroni/post_bootstrap.sh
post_init: /etc/patroni/post_init.sql

postgresql:
listen: 0.0.0.0:5432
connect_address: 192.168.1.100:5432 # THIS node's IP
data_dir: /var/lib/postgresql/16/main
bin_dir: /usr/lib/postgresql/16/bin
config_dir: /etc/postgresql/16/main

authentication:
replication:
username: replicator
password: "StrongReplicaPassword123!"
superuser:
username: postgres
password: "StrongPostgresPassword123!"
rewind: # Used by pg_rewind
username: rewind_user
password: "RewindPassword123!"

# Parameters NOT in DCS (node-specific)

```

```

parameters:
  unix_socket_directories: /var/run/postgresql
  # Node-specific: adjust based on this node's hardware
  # shared_buffers: 8GB # Override cluster default for this large node

pg_hba:
  - local all postgres peer
  - host replication replicator 0.0.0.0/0 scram-sha-256
  - host all all 0.0.0.0/0 scram-sha-256

# pg_basebackup is used to create replica
create_replica_methods:
  - basebackup
basebackup:
  checkpoint: fast
  max-rate: 100M # Limit backup bandwidth
  verbose: true

tags:
  nofailover: false # Set true to exclude this node from failover
candidates
  noloadbalance: false # Set true to exclude from read load balancing
  clonefrom: false # Set true if this node should be preferred source
for cloning
  nosync: false # Set true to always be async (never sync standby)
  # replicatefrom: pg-node2 # Force replication from specific node (cascading)

```

Post-Bootstrap Script

```

#!/bin/bash
# /etc/patroni/post_bootstrap.sh
# Runs after first cluster initialization

PGHOST=/var/run/postgresql
PGUSER=postgres

psql -h $PGHOST -U $PGUSER -d postgres << 'EOF'
-- Create replication user
CREATE ROLE replicator WITH REPLICATION LOGIN PASSWORD 'StrongReplicaPassword123!';

-- Create pg_rewind user
CREATE ROLE rewind_user WITH LOGIN PASSWORD 'RewindPassword123!';
GRANT EXECUTE ON FUNCTION pg_catalog.pg_ls_dir(text, boolean, boolean) TO
rewind_user;
GRANT EXECUTE ON FUNCTION pg_catalog.pg_stat_file(text, boolean) TO rewind_user;
GRANT EXECUTE ON FUNCTION pg_catalog.pg_read_binary_file(text) TO rewind_user;
GRANT EXECUTE ON FUNCTION pg_catalog.pg_read_binary_file(text, bigint, bigint,
boolean) TO rewind_user;

-- Create application user
CREATE ROLE appuser WITH LOGIN PASSWORD 'AppPassword123!';

```

```
CREATE DATABASE appdb OWNER appuser;

-- Extensions
\c appdb
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;
CREATE EXTENSION IF NOT EXISTS pg_trgm;
EOF
```

systemd Service File

```
# /etc/systemd/system/patroni.service
[Unit]
Description=Patroni - PostgreSQL HA
Documentation=https://patroni.readthedocs.io/
After=network.target etcd.service
Wants=etcd.service

[Service]
User=postgres
Group=postgres
Type=simple
ExecStart=/usr/bin/patroni /etc/patroni/patroni.yml
ExecReload=/bin/kill -HUP $MAINPID
KillMode=process
Restart=always
RestartSec=5s

# Logging
StandardOutput=journal
StandardError=journal
SyslogIdentifier=patroni

# Resource limits
LimitNOFILE=65536

[Install]
WantedBy=multi-user.target
```

```
sudo systemctl daemon-reload
sudo systemctl enable patroni
sudo systemctl start patroni
```

patronictl Commands

```
# Configuration file location
export PATRONICTL_CONFIG_FILE=/etc/patroni/patroni.yml
# Or use -c flag: patronictl -c /etc/patroni/patroni.yml <command>
```

```

#-----
# CLUSTER STATUS
#-----

# Show cluster topology
patronictl list

# Example output:
# + Cluster: my-postgres-cluster (7286154547884462247) -----+-----+
# | Member      | Host                | Role    | State  | TL | Lag in MB |
# +-----+-----+-----+-----+-----+-----+
# | pg-node1    | 192.168.1.100:5432 | Leader  | running | 1 |          |
# | pg-node2    | 192.168.1.101:5432 | Replica | running | 1 |          0 |
# | pg-node3    | 192.168.1.102:5432 | Replica | running | 1 |          0 |
# +-----+-----+-----+-----+-----+-----+

# Detailed topology
patronictl topology

# Show cluster history (failover events)
patronictl history

#-----
# FAILOVER AND SWITCHOVER
#-----

# Manual switchover (planned, zero data loss)
patronictl switchover my-postgres-cluster
# Interactive: prompts for candidate, confirmation

# Switchover to specific node
patronictl switchover my-postgres-cluster --master pg-node1 --candidate pg-node2

# Immediate switchover (no confirmation)
patronictl switchover my-postgres-cluster --master pg-node1 --candidate pg-node2 --
force

# Failover (force promote a specific node)
patronictl failover my-postgres-cluster --master pg-node1 --candidate pg-node2 --
force

#-----
# NODE MANAGEMENT
#-----

# Restart PostgreSQL on a node (via Patroni)
patronictl restart my-postgres-cluster pg-node1

# Restart all nodes (rolling)
patronictl restart my-postgres-cluster

# Reload Patroni configuration

```



```

patronictl reload my-postgres-cluster

# Reinitialize a replica (re-clone from primary)
patronictl reinit my-postgres-cluster pg-node2

# Pause automatic failover (manual mode)
patronictl pause my-postgres-cluster

# Resume automatic failover
patronictl resume my-postgres-cluster

#-----
# CONFIGURATION
#-----

# Show current cluster configuration (DCS config)
patronictl show-config my-postgres-cluster

# Edit cluster configuration
patronictl edit-config my-postgres-cluster
# Opens editor, changes are written to DCS and propagated

# Edit specific parameters without editor
patronictl edit-config my-postgres-cluster --set 'loop_wait=15'
patronictl edit-config my-postgres-cluster --set
'postgresql.parameters.max_connections=300'

```

REST API

Patroni exposes a REST API on port 8008 used by HAProxy, monitoring tools, and automation.

```

#-----
# HEALTH CHECK ENDPOINTS (used by HAProxy)
#-----

# Check if node is primary (returns 200 if primary, 503 otherwise)
curl -s http://192.168.1.100:8008/primary
# 200 = yes, this is the primary
# 503 = no, this is not primary

# Check if node is replica (returns 200 if replica)
curl -s http://192.168.1.100:8008/replica
# 200 = yes, replica
# 503 = not a replica

# Check if node is healthy (primary OR replica)
curl -s http://192.168.1.100:8008/health
# 200 = any healthy member

# Async replica check (includes lagging replicas)

```

```

curl -s http://192.168.1.100:8008/read-only
# 200 = available for reads (primary or replica)

#-----
# INFORMATION ENDPOINTS
#-----

# Detailed cluster status (JSON)
curl -s http://192.168.1.100:8008/ | python3 -m json.tool

# Cluster info
curl -s http://192.168.1.100:8008/cluster | python3 -m json.tool

# Configuration
curl -s http://192.168.1.100:8008/config | python3 -m json.tool

# History of leadership changes
curl -s http://192.168.1.100:8008/history | python3 -m json.tool

#-----
# MANAGEMENT ENDPOINTS (POST)
#-----

# Switchover via API
curl -s -X POST http://192.168.1.100:8008/switchover \
  -H "Content-Type: application/json" \
  -d '{"leader": "pg-node1", "candidate": "pg-node2"}'

# Failover
curl -s -X POST http://192.168.1.100:8008/failover \
  -H "Content-Type: application/json" \
  -d '{"leader": "pg-node1", "candidate": "pg-node2"}'

# Reload config
curl -s -X POST http://192.168.1.100:8008/reload

# Restart PostgreSQL
curl -s -X POST http://192.168.1.100:8008/restart \
  -H "Content-Type: application/json" \
  -d '{"restart_pending": true}'

```

Bootstrap and Initialization

```

# Node 1: First node (initializes the cluster)
sudo systemctl start patroni
# Patroni initializes PostgreSQL, creates DCS entries, becomes leader

# Node 2 & 3: Subsequent nodes (join existing cluster)
sudo systemctl start patroni
# Patroni detects existing cluster in DCS

```

```
# Runs pg_basebackup from Node 1
# Starts as replica following Node 1

# Monitor bootstrap
sudo journalctl -u patroni -f

# Expected log on first node:
# INFO: Selected new timeline 1
# INFO: promoted self to leader by acquiring session lock

# Expected log on replica nodes:
# INFO: replica has caught up with the leader
# INFO: Lock owner: pg-node1; I am pg-node2
```

Failover and Switchover

Automatic Failover Flow

1. Primary (pg-node1) stops responding
 2. Patroni on pg-node1 detects DCS connectivity loss → self-demotes PostgreSQL
 3. pg-node2 and pg-node3 detect leader lock expired in DCS
 4. Both attempt to acquire the leader lock
 5. pg-node2 wins (via DCS atomic compare-and-swap)
 6. pg-node2 promotes PostgreSQL to primary
 7. pg-node3 updates primary_conninfo to point to pg-node2
 8. Cluster is healthy again
- Total time: ~30-60 seconds (depending on ttl and loop_wait settings)

Manual Failover Testing

```
# Test automatic failover by stopping Patroni on primary
patronictl list                                # Note current leader
sudo systemctl stop patroni                    # On the primary node
# Wait ~loop_wait + timeout seconds
patronictl list                                # New leader should appear

# Test recovery
patronictl reinit my-postgres-cluster pg-node1 # Reinitialize old primary
sudo systemctl start patroni                   # Start on old primary
patronictl list                                # Should now be replica
```

Monitoring Patroni

```
# Patroni log
sudo journalctl -u patroni -f

# Cluster status check script
```

```
#!/bin/bash
# patroni_monitor.sh

PATRONI_API="http://192.168.1.100:8008"
STATUS=$(curl -s ${PATRONI_API}/cluster)
echo $STATUS | python3 -m json.tool

# Extract leader name
LEADER=$(echo $STATUS | python3 -c "import json,sys; d=json.load(sys.stdin);
print([m['name'] for m in d['members'] if m['role']=='Leader'][0])")
echo "Current leader: $LEADER"

# Check all members
echo $STATUS | python3 -c "
import json, sys
d = json.load(sys.stdin)
for m in d['members']:
    print(f\"{m['name']:15} {m['role']:10} {m['state']:10} lag={m.get('lag',
'N/A')}\")
"
```

```
-- From PostgreSQL side: check patroni-managed cluster
SELECT pg_is_in_recovery() AS is_replica,
       CASE WHEN pg_is_in_recovery() THEN pg_last_wal_replay_lsn()
            ELSE pg_current_wal_lsn()
       END AS current_lsn;
```

Patroni with PgBouncer and HAProxy

See [07_pg_bouncer.md](#) and [08_haproxy_load_balancing.md](#) for detailed configuration. Brief integration overview:

```
Application —> HAProxy (5000: primary, 5001: replica) —> Patroni nodes
                | health check via
                |> /primary endpoint (8008)
                |> /replica endpoint (8008)
```

```
# HAProxy: route to Patroni primary (see 08_haproxy_load_balancing.md for full
config)
backend postgres_primary
    option httpchk GET /primary
    http-check expect status 200
    server pg1 192.168.1.100:5432 check port 8008 inter 2000 fall 3 rise 2
    server pg2 192.168.1.101:5432 check port 8008 inter 2000 fall 3 rise 2
    server pg3 192.168.1.102:5432 check port 8008 inter 2000 fall 3 rise 2
```

Common Mistakes

1. **DCS cluster is not highly available** — 1-node etcd is a SPOF; use 3+ nodes
 2. **TTL too short** — frequent false failovers due to momentary DCS hiccups
 3. **TTL too long** — long failover time on real primary failure
 4. **nofailover: true left on production nodes** — those nodes can't become primary
 5. **Not setting wal_log_hints = on** — pg_rewind fails
 6. **Mismatched cluster scope** — nodes don't see each other
 7. **Not testing failover** — untested automatic failover may not work as expected
 8. **Patroni running as root** — should run as postgres user
 9. **Not monitoring Patroni service** — Patroni process crash goes unnoticed
 10. **Forgetting to handle PgBouncer reload** after failover
-

Best Practices

1. **Always use a 3-node DCS cluster** for production (fault tolerance)
 2. **Set use_pg_rewind: true** for fast re-sync of rejoining nodes
 3. **Use maximum_lag_on_failover** to prevent promoting very stale standbys
 4. **Test failover regularly** with `patronictl switchover`
 5. **Monitor both Patroni and PostgreSQL** separately
 6. **Use tags** to prevent specific nodes from taking leadership (e.g., DR replicas)
 7. **Set reasonable TTL** (30-60s): 30s for fast failover, 60s for fewer false positives
 8. **Version pin patroni** — update carefully in production
-

Performance Considerations

```
# Tuning for performance-sensitive clusters
bootstrap:
  dcs:
    ttl: 30          # Faster failover
    loop_wait: 10    # Patroni checks DCS every 10 seconds
    retry_timeout: 10 # DCS retry timeout
    maximum_lag_on_failover: 10485760 # 10MB max lag for failover
```

Interview Questions

Q1: What is Patroni and why is it preferred for PostgreSQL HA?

A: Patroni is a Python-based HA template for PostgreSQL that automates failover, recovery, and configuration management. It uses a DCS (etcd/Consul) for leader election and cluster state, preventing split-brain through leader lock TTLs. It's preferred because it handles the complex aspects of failover safety (fencing via self-demotion, pg_rewind integration, cascading replica re-pointing) that manual failover is prone to getting wrong.

Q2: How does Patroni prevent split-brain?

A: The primary must continuously renew a TTL-based lock in the DCS. If it can't reach the DCS (e.g., network partition), it cannot renew the lock and self-demotes by stopping PostgreSQL writes (pausing or stopping). Only the node that successfully acquires the DCS lock can promote. This means even if the primary is isolated, it self-demotes before a replica promotes.

Q3: What is `maximum_lag_on_failover` ?

A: It specifies the maximum replication lag (in bytes) a replica can have to be eligible for automatic promotion. If a replica is more than this far behind the failed primary, Patroni skips it as a failover candidate. This prevents promoting a very stale replica that would cause significant data loss.

Q4: What is the difference between `patronictl switchover` and `patronictl failover` ?

A: `switchover` is a planned, zero-data-loss operation: Patroni waits for the candidate to fully catch up with the primary before promoting. `failover` is a forced promotion: it promotes the candidate immediately regardless of replication lag. Use `switchover` for planned maintenance; use `failover` for manual intervention when the primary is confirmed dead.

Q5: How do you add a new node to an existing Patroni cluster?

A: Configure `patroni.yml` with the same `scope` and DCS endpoints, set the node's unique `name` and `connect_address`, then start the Patroni service. Patroni detects the existing cluster in DCS, runs `pg_basebackup` from the current primary, and starts as a replica automatically.

Q6: What does `nofailover: true` tag do?

A: It marks the node as ineligible for automatic promotion. This is useful for nodes that should never become primary: DR replicas in distant data centers (high latency to primary), delayed replicas, or nodes with insufficient hardware.

Q7: How do you perform a rolling restart of PostgreSQL in a Patroni cluster?

A: `patronictl restart <cluster>` performs a rolling restart — it restarts replicas first, then does a `switchover` so the primary restarts last. This ensures minimal downtime. For PostgreSQL config changes requiring restart, Patroni shows `restart_pending` flag and applies them during this rolling restart.

Q8: What is `synchronous_mode` in Patroni?

A: When `synchronous_mode: true`, Patroni manages synchronous replication automatically. It sets `synchronous_standby_names` in PostgreSQL config to point to healthy replicas and removes unreachable ones (preventing primary from blocking when sync standby is down). With `synchronous_mode_strict: true`, Patroni blocks promotions if no synchronous replica is available.

Exercises and Solutions

Exercise 1: Set Up a 3-Node Patroni Cluster

Full setup script:

```
#!/bin/bash
# setup_patroni_cluster.sh
# Run on each node with NODE_IP set to this node's IP

NODE_IP=${1:?Usage: $0 <node-ip>}
NODE_NAME="pg-$(echo $NODE_IP | tr '.' '-')
ETCD_HOSTS="192.168.1.10:2379,192.168.1.11:2379,192.168.1.12:2379"

# Install
pip3 install patroni[etcd3]
```

```
mkdir -p /etc/patroni /var/lib/postgresql/16/main

cat > /etc/patroni/patroni.yml << EOF
scope: my-cluster
namespace: /patroni/
name: ${NODE_NAME}

restapi:
  listen: 0.0.0.0:8008
  connect_address: ${NODE_IP}:8008

etcd3:
  hosts: ${ETCD_HOSTS}

bootstrap:
  dcs:
    ttl: 30
    loop_wait: 10
    retry_timeout: 10
    maximum_lag_on_failover: 1048576
    postgresql:
      use_pg_rewind: true
      parameters:
        wal_level: replica
        max_wal_senders: 10
        max_replication_slots: 10
        wal_log_hints: 'on'
  pg_hba:
    - host replication replicator 0.0.0.0/0 scram-sha-256
    - host all all 0.0.0.0/0 scram-sha-256
  initdb:
    - encoding: UTF8
    - data-checksums

postgresql:
  listen: 0.0.0.0:5432
  connect_address: ${NODE_IP}:5432
  data_dir: /var/lib/postgresql/16/main
  bin_dir: /usr/lib/postgresql/16/bin
  authentication:
    replication:
      username: replicator
      password: ReplicaPass123!
    superuser:
      username: postgres
      password: PostgresPass123!
  rewind:
    username: rewind_user
    password: RewindPass123!
EOF
```

```
echo "Patroni configured for ${NODE_NAME} at ${NODE_IP}"  
echo "Start with: systemctl start patroni"
```

Cross-References

- [02_streaming_replication.md](#) — Streaming replication internals
- [05_failover_procedures.md](#) — Manual failover details
- [07_pgouncer.md](#) — Connection pooling
- [08_haproxy_load_balancing.md](#) — Load balancing configuration
- [09_ha_architecture_patterns.md](#) — Complete HA architectures